

作业4 实现报告

学号：2300012811

姓名：马培杰

Part1: PD Control

实现思路

`part1_cal_torque` 的核心目标是让每个关节尽量跟踪目标姿态。我的实现过程如下：

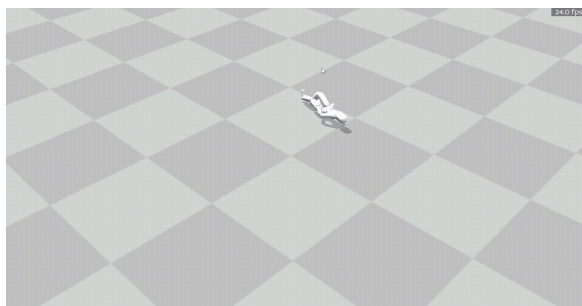
1. 从 `physics_info` 中读取当前 body 朝向、角速度以及父子层级关系。
2. 对每个关节，先计算当前局部旋转：
 - 若当前关节是根关节，则直接使用当前 body 旋转。
 - 若不是根关节，则用父关节旋转的逆乘以当前关节旋转，得到当前局部旋转。
3. 将目标旋转 `pose[i]` 和当前局部旋转做差，使用 `scipy.spatial.transform.Rotation` 计算旋转误差。
4. 将旋转误差转为旋转向量 `rotvec`，再配合相对角速度构造 PD 控制：

$$\tau = k_p e - k_d \dot{e}$$

5. 力矩先在父关节局部坐标系中计算，再变换回全局坐标系。
6. 最后对局部力矩进行 `clip`，避免数值过大导致仿真震荡。

实现效果

实现效果如下：



Part2: Hand of God

实现思路

在 `part2_cal_float_base_torque` 中，我保留了 `part1` 的关节姿态控制，并额外计算根节点的辅助外力和辅助力矩。

具体做法如下：

1. 首先调用 `part1_cal_torque` 得到全身的姿态跟踪力矩。
2. 从 `physics_info` 中读取当前根节点位置 `root_position` 和速度 `root_velocity`。
3. 计算根节点位置误差与速度误差：

$$e_p = p_{target} - p_{root}, \quad e_v = -v_{root}$$

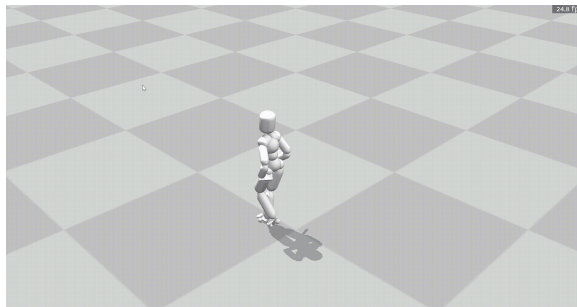
4. 使用 PD 形式生成根节点辅助力：

$$F = k_p e_p + k_d e_v$$

5. 将 `part1` 输出中的根关节力矩作为 `global_root_torque`，同时做适当裁剪。

实现效果

实现效果如下：



Part3: Static Balance

实现过程

这一部分我经历了多轮调试。最开始的版本虽然能提供较强补偿，但会导致角色被持续推着向前走；后续通过加入日志输出和自动评估脚本，对控制行为进行了逐步修正。

最终版本的实现分为三步：

1. 构造站立目标位置

我没有直接把目标根位置固定成 BVH 原始根节点，而是利用双脚支撑区域估计一个更稳定的目标位置。

具体做法是：

1. 取左右脚对应关节的位置。
2. 结合脚部朝向，在脚底前方构造两个支撑点。
3. 使用左右脚关节位置和支撑点的加权平均得到 `target_position`。
4. 竖直方向仍使用 BVH 中初始根节点高度。

这样做的直觉是：角色站立时，根节点应当大致位于双脚支撑区域上方，而不是只跟踪一个单独的骨架根位置。

2. 单独计算基础姿态扭矩

为了避免 part3 与前面跑步/跟踪场景里使用的高增益 part1 参数互相干扰，我在 part3 中重新实现了一套更温和的姿态 PD：

1. 对非根关节，计算当前局部旋转与目标局部旋转之间的误差。
2. 将误差表示为欧拉角差值。
3. 使用较小的姿态控制增益生成基础力矩：
 - 姿态项： $200 * \text{rotation_error}$
 - 阻尼项： $10 * \text{joint_avel}$

之后对过大的扭矩做统一缩放，以避免某个局部关节突然数值爆炸。

3. 将平衡需求映射为下肢关节力矩

根节点的目标平衡需求由

$$F_{virtual} = 4000(p_{target} - p_{root}) - 20v_{root}$$

给出。然后我选择对下肢和根部相关关节：

- RootJoint
- rHip , lHip
- rKnee , lKnee
- rAnkle , lAnkle
- rToeJoint , lToeJoint

施加由叉积得到的补偿力矩：

$$\tau_i \leftarrow \tau_i - (r_i \times F_{virtual})$$

其中 r_i 为根节点到第 i 个关节的力臂。这样就相当于用下肢关节力矩来模拟原本作用在根节点上的稳定作用。

调试过程

为了提高 part3 的稳定性，我增加了两种调试手段：

1. 在 `answer_task1.py` 中定期把中间变量写入 `part3_debug.log`。
2. 新建了 `eval_part3.py`，可以在 `conda` 的 `MoCCA` 环境下自动运行仿真，并在角色摔倒或水平漂移过大时提前停止。

自动评估脚本记录的指标包括：

- 仿真持续帧数
- 是否摔倒
- 是否漂移超阈值
- 最大高度下降
- 最大水平漂移

最终结果

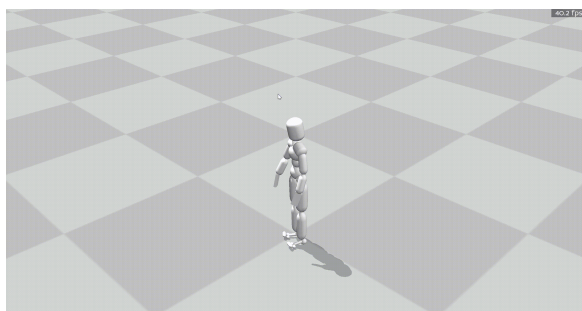
最终版本在自动评估脚本中的结果为：

- `frames = 1855`
- `fell = False`
- `drifted = True`
- `max_height_drop = 0.0089`
- `max_planar_drift = 0.4505`

这说明：

1. 角色在较长时间内没有直接摔倒。
2. 高度变化非常小，说明竖直方向支撑比较稳定。
3. 最终停止的原因主要是长时间扰动下出现了累计水平漂移，而不是快速失稳。

实现效果如下：



在可视化仿真中，角色已经能够在随机扰动下维持较长时间站立，整体效果达到了基本要求。

遇到的问题与解决方法

1. 姿态控制和站立平衡之间互相干扰

一开始直接复用高增益的 part1 力矩，虽然姿态跟踪很强，但在站立场景下容易和下肢平衡补偿相互冲突，导致角色前冲或翻倒。后续通过在 part3 中单独使用更温和的姿态 PD，问题得到缓解。

2. 目标根位置设置不合理

如果直接把根节点目标位置放在脚部支撑点附近，角色会持续朝那个方向移动，表现为“越站越往前走”。因此最后采用了脚部位置与脚底支撑点的加权平均，并保持竖直高度稳定。

3. 平衡补偿过强或过弱

- 补偿过强时，角色容易被掀翻或者快速前冲。
- 补偿过弱时，角色会慢慢下沉、屈膝并最终倒地。

因此最终参数是在多轮测试中折中得到的。